Configuration

Version: 1.0

Purpose

This document defines the configuration architecture of Trading Platform Pro and its primary application, the Trading Cockpit.
The configuration system provides a centralized, deterministic and validated mechanism for managing application settings.
Configuration shall remain separate from business logic.
Trading environment, broker and market data configuration are operationally critical and require explicit handling.

Objectives

The configuration system shall provide:
• centralized configuration management
• deterministic configuration loading
• explicit environment selection
• startup validation
• strong typing
• secure secret handling
• explicit PAPER and LIVE separation
• observable configuration state
• testable configuration behaviour

Configuration Principles

Configuration shall be:
• externalized
• explicit
• validated
• strongly typed
• deterministic
• minimal
• documented
• independent from business logic
Configuration shall not contain trading decisions or domain business rules.
Configuration shall not silently change application behaviour through undocumented defaults.

Configuration Ownership

Configuration defines technical and operational settings.
Examples:
• application behaviour
• runtime behaviour
• logging
• persistence
• broker connectivity
• market data connectivity
• workspace persistence
• monitoring
• technical timeouts
Business rules belong to Domain or Application capabilities.

Examples of values that shall not be hidden as generic technical configuration without explicit product ownership include:
• candidate acceptance rules
• trading decision rules
• portfolio risk limits
• strategy selection rules
Business-controlled parameters require explicit capability ownership.

Configuration Sources

Supported configuration sources may include:
1. Default application configuration
2. YAML configuration files
3. Environment profile configuration
4. Environment variables
5. Command-line arguments
6. Explicit runtime overrides where supported
Every supported source shall have a defined responsibility.
Avoid multiple configuration sources for the same value without a defined precedence rule.

Configuration Precedence

Configuration precedence shall be deterministic.
Default precedence from lowest to highest:
```text
Application Defaults
↓
Base YAML Configuration
↓
Environment Profile
↓
Environment Variables
↓
Command-Line Arguments
↓
Explicit Runtime Override
```
A higher-precedence source may override a lower-precedence source only where the configuration field supports overriding.
Runtime overrides shall not be available for operationally critical values unless explicitly designed.

Configuration Profiles

Initial configuration profiles may include:
• development
• testing
• paper
• live
Profile names shall use lowercase identifiers.
The active profile shall be explicit.
The application shall not infer LIVE mode from broker connectivity alone.

Development Profile

The `development` profile supports local application development.

Typical characteristics may include:
• development logging
• local persistence
• development integrations
• diagnostic capabilities
Development configuration shall not imply LIVE trading capability.

Testing Profile

The `testing` profile supports automated tests.
Typical characteristics may include:
• deterministic configuration
• temporary persistence
• fake adapters
• controlled clocks
• simulated broker responses
• simulated market data
Tests shall not require LIVE broker connectivity.

PAPER Profile

The `paper` profile supports broker paper-trading workflows.
PAPER configuration shall remain explicitly distinguishable from LIVE configuration.
Typical PAPER settings may include:
• PAPER broker endpoint
• PAPER account context
• PAPER-specific connection parameters
• PAPER operational indicators
The Trading Cockpit shall visibly expose PAPER execution context where trading functionality is available.

LIVE Profile

The `live` profile supports LIVE trading workflows.
LIVE configuration is operationally critical.
LIVE mode shall:
• require explicit profile selection
• use explicit LIVE broker configuration
• remain distinguishable from PAPER
• be visible in the Trading Cockpit
• fail startup validation when critical LIVE configuration is invalid
LIVE trading shall never be activated through an implicit fallback.
The application shall not automatically switch from PAPER to LIVE.

PAPER and LIVE Separation

PAPER and LIVE configuration shall remain explicitly separated.
Example:
```text
config/
■
■■■ base.yaml
■■■ development/
■■■ testing/
■■■ paper/
```

■■■ live/
```
The exact physical configuration structure may evolve with implementation requirements.
PAPER and LIVE shall not share operationally critical broker settings through ambiguous configuration.
Shared technical defaults may remain in base configuration.
---
# Active Trading Environment
The active trading environment shall use an explicit typed value.
Example:
```python
class TradingEnvironment(Enum):
PAPER = "paper"
LIVE = "live"
```
Avoid loosely defined values such as:
```python
"prod"
"real"
"active"
```
Application workflows shall be able to determine the active trading environment explicitly.

Configuration Loading
The startup configuration sequence shall:
1. Determine the requested profile.
2. Load application defaults.
3. Load base configuration.
4. Load profile configuration.
5. Apply supported environment variables.
6. Apply supported command-line arguments.
7. Apply explicit runtime overrides where supported.
8. Build the typed configuration model.
9. Validate the final configuration.
10. Register configuration in the dependency container.
Application workflows shall not start before configuration validation succeeds.

Fail-Fast Validation
Invalid critical configuration shall fail startup.
Examples:
• unknown profile
• invalid trading environment
• missing required broker configuration
• invalid database path
• invalid numeric range
• invalid timeout
• incompatible configuration values
The application shall report the affected configuration capability.
Secrets shall not be included in validation error output.

Strong Typing
```

Application components shall consume typed configuration models.
Prefer:
```python
@dataclass(frozen=True)
class BrokerConfiguration:
host: str
port: int
client_id: int
environment: TradingEnvironment
```
Avoid passing generic configuration dictionaries through the application.
Avoid:
```python
config["broker"]["port"]
```
inside Domain or Application business logic.
Typed configuration shall be created at the configuration boundary.

Configuration Immutability

Validated startup configuration should be immutable where practical.
Application components shall not mutate shared configuration objects.
Runtime-changeable settings require explicit ownership and lifecycle rules.
A mutable application preference is not automatically startup configuration.

Configuration Structure

Configuration shall be organized by responsibility.
Initial sections may include:
• Application
• Runtime
• Logging
• Database
• Broker
• Market Data
• Workspace
• Messaging
• Scheduler
• Monitoring
• Health
• Security
Only implemented capabilities require configuration sections.
Do not create empty configuration sections for hypothetical future capabilities.

Application Configuration

Application configuration may include:
• application identity
• active profile
• feature availability
• UI startup behaviour
Application configuration shall not contain domain business rules.

Runtime Configuration

Runtime configuration may include:
• shutdown timeout
• worker lifecycle settings
• runtime health intervals
• controlled technical timeouts
Runtime values shall preserve deterministic lifecycle behaviour.

Logging Configuration

Logging configuration may include:
• log level
• output destination
• structured logging format
• retention settings where supported
Sensitive information shall not be enabled through logging configuration.
Debug logging shall not expose secrets.

Database Configuration

Database configuration may include:
• database location
• connection settings
• initialization behaviour
The initial Trading Cockpit uses SQLite.
Database configuration shall remain isolated from Domain logic.

Broker Configuration

Broker configuration may include:
• provider
• host
• port
• client identifier
• account context
• connection timeout
• active trading environment
Broker configuration shall explicitly preserve PAPER or LIVE context.
Broker credentials shall not be stored in source-controlled YAML files.
Provider-specific settings belong to provider-specific configuration models where required.

Market Data Configuration

Market data configuration may include:
• provider
• connection settings
• subscription settings
• technical refresh intervals
• technical timeout values
Market data configuration shall not contain trading decision rules.
Data freshness thresholds require explicit product or application ownership when they affect trading workflows.

Workspace Configuration

Workspace configuration may include:

• workspace storage location
• startup workspace behaviour
• restoration behaviour
• fallback workspace
User workspace state is not static application configuration.
Workspace layouts and widget state shall remain in workspace persistence.

Messaging Configuration

Messaging configuration may include:
• internal queue limits
• technical delivery settings
• event processing settings
Messaging configuration shall not define business workflow decisions.

Scheduler Configuration

Scheduler configuration may include technical scheduling behaviour.
Examples:
• maintenance intervals
• health-check intervals
• background refresh intervals
Trading workflow schedules require explicit Application or Domain ownership.
The scheduler shall not define trading strategy timing through generic infrastructure configuration.

Monitoring Configuration

Monitoring configuration may include:
• enabled health metrics
• collection intervals
• operational thresholds
Monitoring shall observe application behaviour.
Monitoring configuration shall not alter trading decisions.

Health Configuration

Health configuration may include:
• check intervals
• technical timeout values
• enabled health checks
Health state definitions shall remain consistent with the runtime and monitoring architecture.

Security Configuration

Security configuration may include:
• secret provider selection
• credential storage integration
• security-related technical settings
Security configuration shall not expose secret values through diagnostics or UI configuration views.

Secrets

Sensitive information shall never be stored directly in source code.
Examples:
• API keys

• access tokens
• passwords
• broker credentials
Secrets shall not be committed to source control.
Secrets should be provided through approved external mechanisms such as:
• environment variables
• operating system credential storage
• dedicated secret providers
The selected mechanism shall be documented when implemented.

Secret Access

Secret access shall remain explicit.
Application components should receive required credentials through controlled configuration or credential abstractions.
Avoid unrestricted global secret access.
Secrets shall not be included in:
• logs
• exceptions
• UI error messages
• diagnostic exports

Environment Variables

Environment variables should be reserved primarily for:
• secrets
• deployment-specific values
• machine-specific technical configuration
Environment variable names shall use a consistent prefix.
Example:
```text
TRADING_PLATFORM_BROKER_HOST
TRADING_PLATFORM_BROKER_PORT
```
Environment variables shall be parsed and validated before use.

Command-Line Arguments

Command-line arguments may select or override explicitly supported startup behaviour.
Examples:
```text
--profile paper
--profile live
```
Operationally critical command-line overrides shall remain visible in startup logs without exposing secrets.
Unsupported arbitrary configuration overrides shall not be accepted silently.

Runtime Overrides

Runtime overrides require explicit design.
A runtime override shall define:
• owning capability
• allowed values
• persistence behaviour

• validation
• operational impact
LIVE-critical configuration shall not be changed through unrestricted runtime overrides.
Changing broker execution environment during an active application session is prohibited unless explicitly designed and approved.

Configuration Observability

The application shall expose relevant non-sensitive configuration context.
Examples:
• active profile
• PAPER or LIVE environment
• configured broker provider
• configured market data provider
• database mode
Startup logging shall record relevant configuration context.
Secret values shall never be logged.

Configuration and UI

Operationally important configuration state shall be visible in the Trading Cockpit where relevant.
Examples:
• PAPER mode
• LIVE mode
• broker provider
• broker connection state
• market data provider
The UI shall not require users to infer LIVE execution context from technical connection details.

Configuration Compatibility

Configuration compatibility shall be evaluated based on real application requirements.
Breaking configuration changes require:
• documentation updates
• migration guidance where persisted configuration is affected
• automated tests
• explicit default handling
Do not preserve unused configuration fields solely for hypothetical backward compatibility.

Unused Configuration

Unused configuration entries shall be removed.
The application shall avoid configuration flags that no longer affect behaviour.
Deprecated configuration shall have:
• documented replacement
• defined removal plan
Configuration shall not become an archive of historical options.

Testing

Configuration behaviour requires automated tests.
Tests shall verify where relevant:
• precedence
• profile selection
• typed parsing

• required values
• invalid values
• environment variable overrides
• command-line overrides
• PAPER and LIVE separation
• missing secrets
• secret redaction
• incompatible values
• fail-fast behaviour
Tests shall remain deterministic.

Configuration Governance
Configuration changes shall:
• support a defined technical or product requirement
• preserve deterministic precedence
• remain strongly typed
• remain validated
• evaluate PAPER and LIVE impact
• evaluate secret handling
• update automated tests
• update documentation
Configuration is part of the system architecture.

Configuration Review Checklist
Before introducing or changing configuration verify:
• owning capability identified
• configuration requirement identified
• business rule not hidden in technical configuration
• source identified
• precedence defined
• type defined
• default behaviour defined
• validation defined
• immutability evaluated
• runtime mutability evaluated
• PAPER impact evaluated
• LIVE impact evaluated
• secret handling evaluated
• operational visibility evaluated
• automated tests added
• documentation synchronized

Related Documents
• Product_Vision.md
• Product_Roadmap.md
• Project_Overview.md
• Architecture.md
• Infrastructure.md
• Technical_Specifications.md
• API_Guidelines.md
• Runtime.md
• Logging.md
• Monitoring.md

- Security.md
- Testing_Strategy.md
- AGENTS.md